Lab#6

Implementation of linked list with the help of algorithms

for Insertion, Deletion and Search an element

LINKED LIST:

Linked List is a sequence of links which contains items. Each link contains a connection to another link. Linked list is the second most used data structure after array. ′ Following are the important terms to understand the concept of Linked List.

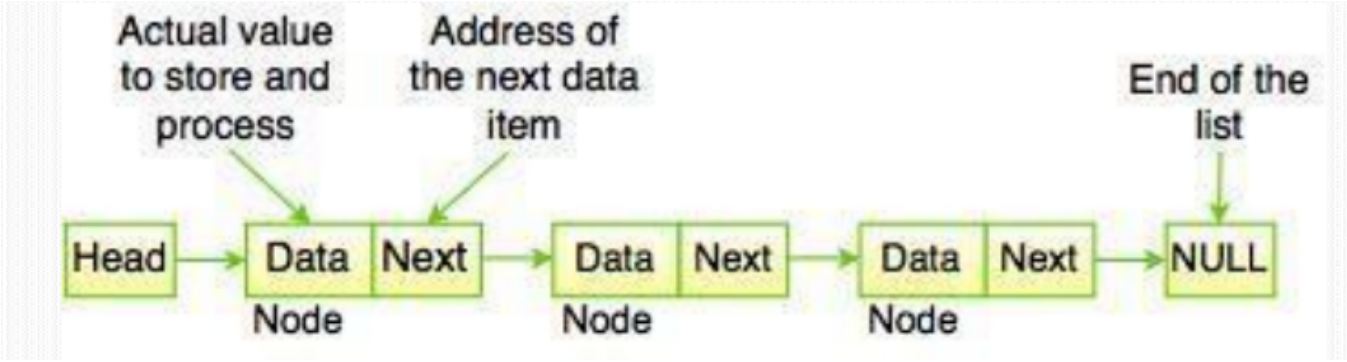
Link − Each link of a linked list can store a data called an element.

Next − Each link of a linked list contains a link to the next link called Next.

Linked List − A Linked List contains the connection link to the first link called First.

As per the following illustration, following are the important points to be considered.

- Linked List contains a link element called first.
- Each link carries a data field(s) and a link field called next.
- Each link is linked with its next link using its next link.
- Last link carries a link as null to mark the end of the list.



EXAMPLES OF LINKED LIST

A goods train is the best example of singly linked list where the head of the train is the head of the link list linking to the next cell and each cell containing the goods is the value of that cell means of that node and each cell linking to the next cell and the last node has no next cell means it's linking to N U L L.

History of web browser. It creates a linked list of web-pages visited, so that when you check history (traversal of a list) or press back button, the previous node's data is fetched.

ARRAY VS LINKED LIST

Array	Linked List
An array is a list store in contiguous memory.	A linked list is a list scattered throughout memory and connected with pointers/Links of next element.
Any element of an array can be accessed quickly.	The elements of a linked list must be accessed in order. Linear Access Mechanism
The size of an array is fixed.	The length of the list can change at any time, making it a dynamic data structure

OPERATIONS ON LINKED LIST

Following are the basic operations supported by a list.

- Insertion − Adds an element in the list.
- Traverse − Access all the elements in the list.
- Deletion − Deletes an element at the beginning of the list.
- Display − Displays the complete list.
- Search − Searches an element using the given key.
- Delete − Deletes an element using the given key

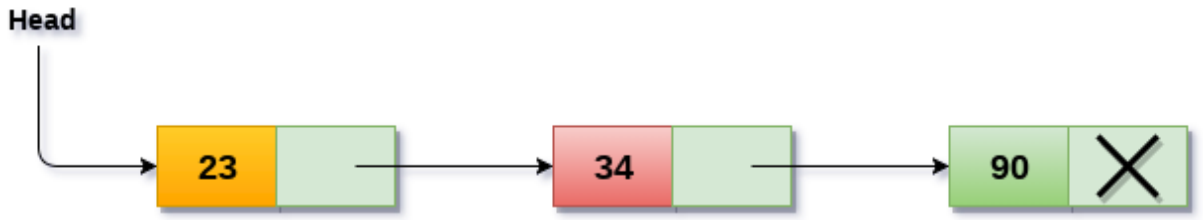
TYPES OF LINKED LIST

Following are the various types of linked list.

1. Simple Linked List
2. Doubly Linked List
3. Circular Linked List

SIMPLE /SINGLY LINKED LIST

A **singly linked list** is a type of linked list that is *unidirectional*, that is, it can be traversed in only one direction from head to the last node (tail).
Undo button of any application like Microsoft Word, Paint, etc. A linked list of states is example of singly linked list.



TRAVERSING IN SINGLY LINKED LIST

1. Set PTR := START
2. Repeat Steps 3-4 while PTR ≠ N U L L
3. Apply process to INFO[PTR]
4. Set PTR:= LINK[PTR] [end of loop]
5. Exit

EXAMPLE (TRAVERSING)

- Traverse the following Linked List

Solution:

PTR : = Start = 300
While PTR ≠ NULL
300 ≠ NULL
INFO[300]= A
PTR : = LINK[300] = 100
While PTR ≠ NULL
100 ≠ NULL
INFO[100] = C
PTR : = LINK[100] =900

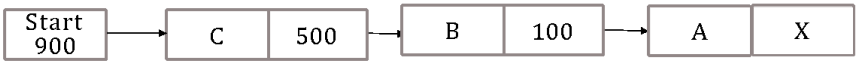
While PTR ≠ NULL
900 ≠ NULL
INFO[900] = D
PTR : = LINK[900] = X
PTR : = X
Exit

SEARCHING IN SINGLY LINKED LIST

1. Set PTR := START
2. Repeat Steps 3 while PTR ≠ N U L L
3. If ITEM = INFO[PTR] then
   Set LOC: =PTR and Exit Else
   Set PTR:= LINK[PTR]
   [End of if Structure]
   [end of loop]
4. [Search is Unsuccessful] Set LOC: = Null
5. Exit

EXAMPLE

- Search A from the given Linked List



- Solution
- ITEM : = A
- PTR : = 900
- While PTR ≠ NULL
- 900 ≠ NULL
- ITEM : = INFO[900]
- A ≠ C
- Else
- PTR : = LINK [PTR] = LINK [500]
- PTR:= 500
- While PTR ≠ NULL
- 500 ≠ NULL
- ITEM : = INFO [500]
- A = B
- Else
- PTR : = LINK[PTR] = LINK [100]= 100
- While PTR ≠ NULL
- 100 ≠ NULL
- ITEM : = INFO[100]
- A = =A
- LOC : = PTR
- LOC = 100
- EXIT

MEMORY ALLOCATION

Whenever a new node is created, memory is allocated by the system. This memory is taken from list of those memory locations which are free i.e. not allocated. This list is called AVAIL List. Similarly, whenever a node is deleted, the deleted space becomes reusable and is added to the list of unused space i.e. to AVAIL List. This unused space can be used in future for memory allocation.

Memory allocation is of two types-

1. Static Memory Allocation (Compilation time- Fixed Memory)
2. Dynamic Memory Allocation (Running/Execution time- Not Fixed memory)

OVERFLOW AND UNDERFLOW

Overflow happens at the time of insertion. If we have to insert new space into the data structure, but there is no free space i.e. availability list is empty, then this situation is called 'Overflow'. The programmer can handle this situation by printing the message of OVERFLOW.

Underflow happens at the time of deletion. If we have to delete data from the data structure, but there is no data in the data structure i.e. data structure is empty, then this situation is called 'Underflow'. The programmer can handle this situation by printing the message of UNDERFLOW.

INSERTION IN SINGLY LINKED LIST

INSERTING THE ELEMENT AT BEGINNING

1. If AVAIL := Null
   Then Write Overflow and Exit
2. Set NEW: =AVAIL and AVAIL:= Link[AVAIL]
3. Set INFO[NEW]:= ITEM
4. Set LINK[NEW]:=START
5. Set START: =NEW
6. Exit

INSERTING THE ELEMENT AT ANY POSITION

- **INSLOC[INFO, LINK, START, AVAIL, LOC, ITEM**

1. If AVAIL=NULL then write OVERFLOW and exit.
2. Set NEW: =AVAIL and AVAIL:=LINK[AVAIL]
3. Set INFO[NEW]:=ITEM[copies new data into new node]

4. If LO C = N U L L
   then set LINK[NEW]:=START and START:=NEW
   Else [insert after node with location Loc]
   Set LINK[NEW]:=LINK[LOC] and LINK[LOC]: =NEW
   [end of if structure]
5. Exit

INSERTING THE ELEMENT AT END

- **INLAST(INFO, START, AVAIL, ITEM)**

1. If AVAIL=NULL
Then write OVERFLOW and exit.
2.[remove first node from AVAIL list] set NEW=AVAIL and AVAIL=LINK[AVAIL]
3. Set INFO[NEW] =ITEM [copies new data into new node]
4. If START=NULLSet START=NEW and LOC=START
5. Repeat step 6 untill LINK[LOC] != N U L L
6. Set LOC=LINK[LOC]
7. Set LINK[LOC] =NEW
8. Exit

DELETION IN SINGLY LINKED LIST

DELETE FIRST ELEMENT OF THE LIST

- **Delete(INFO, LINK, START, ITEM, LOC, LOCP)**

1. If START = Null Then Set
2. If L O C = N U LL,
   Then Write: ITEM not in list, and Exit.
3. [Delete node] If LO C P = N U LL,
   then: Set START = LINK[START] [Deletes first node]
   Else Set LINK[LOCP] = LINK[LOC] [End of If structure]
4. [Return deleted node to the AVAIL list] Set LINK[LOC] = AVAIL and AVAIL = LOC.
5. Exit

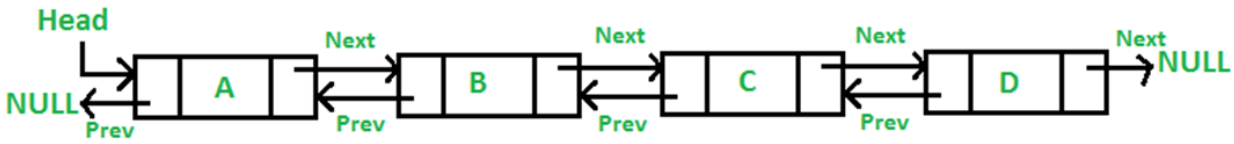
DELETING ANY ELEMENT OF THE LIST

1. If START = N U LL,
   Then: Set L O C = N U L L and LOCP = NULL, and Return. [End of If structure]
2. [ITEM in first node?] If INFO[START] = ITEM, then: Set LOC = START and LOCP = NULL, and Return. [End of If structure]
3. Set SAVE = START and PTR = LINK[START]
   [Initializes pointers]
4. Repeat steps 5 and 6 while PTR ≠ N U L L
5. If INFO[PTR] = ITEM, then Set L O C = PTR and LO C P = SAVE, and Return
   [End of If structure]

6. Set Save = PTR and PTR = LINK[PTR] [Updates pointers] [End of step 4 loop]
7. Set L O C = N U L L [Search unsuccessful]
8. Return

DOUBLY LINKED LIST

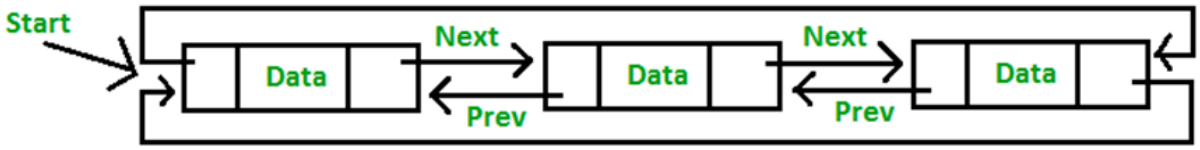
- Singly linked list only allows us to traverse forward, i.e., we cannot iterate back to a previous node if required.
- A doubly linked list contains a pointer to the next node as well as the previous node. This ensures that the list can be traversed in both directions.
- Examples of doubly linked list
  - A music player which has next and previous buttons.
  - Undo Redo operations
  - The Browser cache which allows you to move front and back between pages is also a good example of doubly linked list.
- Double linked list node has three fundamental members:
  1. The data
  2. A pointer to the next node
  3. A pointer to the previous node



CIRCULAR LINKED LIST

Circular linked list is a linked list where all nodes are connected to form a circle. There is no NULL at the end. A circular linked list can be a singly circular linked list or doubly circular linked list.

Any node can be a starting point. We can traverse the whole list by starting from any point. We just need to stop when the first visited node is visited again.



LAB TASKS

1. Write a C++ program to create and display a singly linked list. The program should allow the user to input data for the list, and then display the list in the order entered.

2. Write a C++ program to create a singly linked list of n nodes and count the number of nodes.

Original Linked list:
13 11 9 7 5 3 1
Number of nodes in the said Linked list:
7

3. Write a C++ program to insert a new node at the beginning of a Singly Linked List.

Original Linked list:
13 11 9 7 5 3 1
Insert a new node at the beginning of a Singly Linked List:
0 13 11 9 7 5 3 1

4. Write a C++ program to delete the n^{th} node of a Singly Linked List from the end.

Original list:
7 5 3 1
Remove the 2nd node from the end of the said list:
Updated list:
7 5 1
Remove the 3rd node from the end of the said list:
Updated list:
5 1

5. Write a C++ program to create and display a doubly linked list.

Doubly linked list is as follows:

Traversal in Forward direction:
Orange White Green Red
Traversal in Reverse direction:
Red Green White Orange

6. Write a C++ program to insert a new node at the end of a Doubly Linked List.

Doubly linked list is as follows:
Traversal in Forward direction: Orange White Green Red
Traversal in Reverse direction: Red Green White Orange
Insert a new node at the end of a Doubly Linked List:
Traversal in Forward direction: Orange White Green Red Pink
Traversal in Reverse direction: Pink Red Green White Orange